

NUBOM

Ni Un Bocado Menos

Documentación de Base de Datos y API REST

Proyecto final · Módulo 7 · Diplomado en Desarrollo de Software
Spring Boot 3.5 · Spring Data JPA · MariaDB
Armando Luna
Septiembre 2026

Contenido

1. Introducción al sistema
2. Descripción general de la base de datos
 - 2.1 Motor y estrategia de generación del esquema
 - 2.2 Tablas del sistema
 - 2.3 Relaciones entre entidades (incluye 2.3.1 relación N:M)
 - 2.4 Diagramas entidad-relación
3. Arquitectura de la API REST
4. Manejo global de errores y serialización JSON
5. Documentación de las operaciones de la API REST
 - 5.1 Recurso Hogar
 - 5.2 Recurso Inventario
 - 5.3 Recurso Producto
 - 5.4 Recurso Lista de compras
 - 5.5 Catálogo de tipos de inventario
 - 5.6 Recurso Usuario (incluye relación N:M)
 - 5.7 Catálogo de roles
6. Evidencia de pruebas (curl) y escenarios exigidos

1. Introducción al sistema

NUBOM (“Ni Un Bocado Menos”) es una aplicación Spring Boot desarrollada a lo largo del diplomado para administrar el inventario del hogar (refrigerador y alacena) y la lista de compras de una familia. La aplicación original expone una interfaz web con Thymeleaf, autenticación por sesión y persistencia con Spring Data JPA sobre MariaDB.

Para este entregable del Módulo 7 se integró sobre ese mismo proyecto una **API RESTful** independiente (paquete `dgtic.core.rest`) que expone las mismas entidades del dominio a través de operaciones CRUD estándar, utilizando DTO, una clase Mapper por recurso, `ResponseEntity` para personalizar las respuestas HTTP y un manejador global de errores con `@RestControllerAdvice`. La API convive con la aplicación Thymeleaf existente sin afectarla: sus rutas (`/api/**`) quedan excluidas del filtro de sesión que protege las páginas HTML.

Como parte de este trabajo también se reforzaron las entidades JPA para usar relaciones reales (`@OneToMany` / `@ManyToOne`) en lugar de columnas planas de llave foránea, de modo que las relaciones 1:N queden correctamente implementadas y sean utilizadas tanto por la aplicación web como por la API REST.

La API expone ahora **las 8 tablas principales** del modelo de datos, con al menos una relación 1:N (varias, de hecho: hogar→inventario, inventario_tipo→inventario, inventario→producto, hogar→lista_compra) y una relación **N:M** (usuario↔hogar, a través de la tabla intermedia usuario_hogar, con el rol como atributo propio de la asociación), ambas correctamente implementadas con JPA y usadas en operaciones reales de la API (crear, consultar y eliminar información asociada desde cualquiera de los dos lados).

2. Descripción general de la base de datos

2.1 Motor y estrategia de generación del esquema

Motor: **MariaDB** (base de datos nubom). El esquema se genera automáticamente al iniciar la aplicación mediante Hibernate (`spring.jpa.hibernate.ddl-auto=create`) a partir de las anotaciones JPA de las entidades, y se siembra con datos de prueba desde `data.sql` gracias a `spring.jpa.defer-datasource-initialization=true`. El archivo `esquema-base-datos.sql`, incluido en el proyecto, documenta ese mismo esquema (incluyendo las llaves foráneas reales generadas por las relaciones JPA) para consulta manual.

2.2 Tablas del sistema

Tabla	Propósito	Columnas principales
hogar	Representa un hogar/familia dentro de la aplicación. Es la raíz de la relación 1:N con inventario y con lista_compra: al eliminarse, sus inventarios (y los productos de estos) y su lista de compras se eliminan en cascada.	id_hogar (PK), nombre, fecha_creacion
inventario_tipo	Catálogo de los tipos de inventario disponibles (Refrigerador, Alacena). Es el lado "1" de una relación 1:N con inventario.	id_tipo (PK), nombre, descripcion, icono
inventario	Un espacio de almacenamiento (refrigerador o alacena) perteneciente a un hogar, con una capacidad máxima expresada en número de objetos. Es el lado "N" de hogar->inventario y el lado "1" de inventario->producto.	id_inventario (PK), id_hogar (FK), id_tipo (FK), nombre, capacidad_maxima, fecha_creacion
producto	Un producto guardado dentro de un inventario específico (leche, arroz, etc.). Es el lado "N" de la relación inventario->producto.	id_producto (PK), id_inventario (FK), nombre, cantidad, unidad, categoria, fecha_ingreso, activo
lista_compra	Un producto pendiente (o ya comprado) en la lista de compras de un hogar. Es el lado "N" de la relación hogar->lista_compra.	id_item (PK), id_hogar (FK), nombre, categoria, cantidad, unidad, comprado, fecha_creacion
rol	Catálogo de roles que un usuario puede tener dentro de un hogar (PROPIETARIO, FAMILIAR). Es el atributo propio de la relación N:M usuario<->hogar.	id_rol (PK), nombre, descripcion
usuario	Cuenta de acceso a la aplicación (login/registro web, y CRUD vía API REST). Es uno de los dos lados de la relación N:M con hogar.	id_usuario (PK), nombre, correo (único), contrasena, fecha_registro, activo
usuario_hogar	Tabla intermedia de la relación N:M usuario<->hogar: indica a qué hogar(es) pertenece cada usuario y con qué rol en cada uno. Al llevar un atributo propio (id_rol, fecha_union, activo) se modela como entidad de unión con @ManyToOne hacia los tres catálogos, en vez de un simple @ManyToMany.	id_usuario_hogar (PK), id_usuario (FK), id_hogar (FK), id_rol (FK), fecha_union, activo

2.3 Relaciones entre entidades

Las relaciones 1:N centrales del sistema se implementaron con anotaciones JPA reales (@OneToMany del lado “1” con mappedBy, y @ManyToOne + @JoinColumn del lado “N”), en lugar de manejar los identificadores foráneos como enteros sueltos. Esto permite que Hibernate genere llaves foráneas reales en la base de datos y que las colecciones se usen directamente desde los servicios y la API REST.

Relación	Descripción / regla de negocio
hogar (1) — inventario (N)	Un hogar tiene varios inventarios (típicamente Refrigerador y Alacena). cascade=ALL, orphanRemoval=true: al borrar el hogar se borran sus inventarios.
inventario_tipo (1) — inventario (N)	Un tipo de inventario (p. ej. “Alacena”) puede estar asignado a varios inventarios de distintos hogares. Sin cascada: no se permite borrar un tipo en uso (409 Conflict).
inventario (1) — producto (N)	Un inventario contiene varios productos. cascade=ALL, orphanRemoval=true: al borrar el inventario se borran sus productos.
hogar (1) — lista_compra (N)	Un hogar tiene varios items en su lista de compras. cascade=ALL, orphanRemoval=true.
usuario (N) <-> hogar (N)	Relación N:M: un usuario puede pertenecer a varios hogares (a lo largo del tiempo) y un hogar tiene varios usuarios, con el rol como atributo propio de cada asociación. Se modela con la tabla intermedia usuario_hogar (ver 2.3.1). Sin cascada: borrar un usuario o un hogar con membresías activas se rechaza por la llave foránea (409 Conflict).

2.3.1 Relación N:M usuario <-> hogar (tabla intermedia)

A diferencia de las relaciones 1:N anteriores, usuario y hogar se relacionan en ambos sentidos: un usuario puede tener membresías en varios hogares y un hogar agrupa a varios usuarios. Como la asociación lleva un atributo propio (el rol, la fecha de unión y si sigue activa), se modela con una **entidad de unión** (usuario_hogar) que tiene @ManyToOne hacia los tres catálogos, en vez de un simple @ManyToMany — la práctica recomendada por JPA/Hibernate para relaciones N:M "con carga".

```

@Entity(name = "usuario_hogar")
@Table(name = "usuario_hogar")
public class UsuarioHogarBd {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer idUsuarioHogar;

    // Lado "N" hacia usuario de la relacion N:M usuario <-> hogar.
    @JsonIgnore
    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "id_usuario", nullable = false)
    private UsuarioBd usuario;

    // Lado "N" hacia hogar de la relacion N:M usuario <-> hogar.
    @JsonIgnore
    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "id_hogar", nullable = false)
    private HogarBd hogar;

    // Rol del usuario dentro de ESE hogar (atributo propio de la asociacion).
    @JsonIgnore
    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "id_rol", nullable = false)
    private RolBd rol;

    private LocalDateTime fechaUnion;
    private Boolean activo;
}

```

El repositorio consulta ambos lados navegando las relaciones (sin necesidad de un JOIN manual):

`findByUsuario_IdUsuarioAndActivoTrue(idUsuario)` (lado usuario) y

`findByHogar_IdHogarAndActivoTrue(idHogar)` (lado hogar).

Ejemplo de implementación (lado "1" en HogarBd):

```

@Entity(name = "hogar")
@Table(name = "hogar")
public class HogarBd {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer idHogar;

    private String nombre;
    private LocalDateTime fechaCreacion;

    // Lado "1" de la relacion 1:N hogar -> inventario.
    @OneToMany(mappedBy = "hogar", cascade = CascadeType.ALL, orphanRemoval = true)
    private List<InventarioBd> inventarios = new ArrayList<>();

    @OneToMany(mappedBy = "hogar", cascade = CascadeType.ALL, orphanRemoval = true)
    private List<ListaCompraBd> listaCompra = new ArrayList<>();
}

```

Lado "N" correspondiente (InventarioBd):

```

@Entity(name = "inventario")
@Table(name = "inventario")
public class InventarioBd {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer idInventario;

    // Lado "N" de la relacion 1:N hogar -> inventario.
    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "id_hogar", nullable = false)
    private HogarBd hogar;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "id_tipo", nullable = false)
    private InventarioTipoBd tipo;

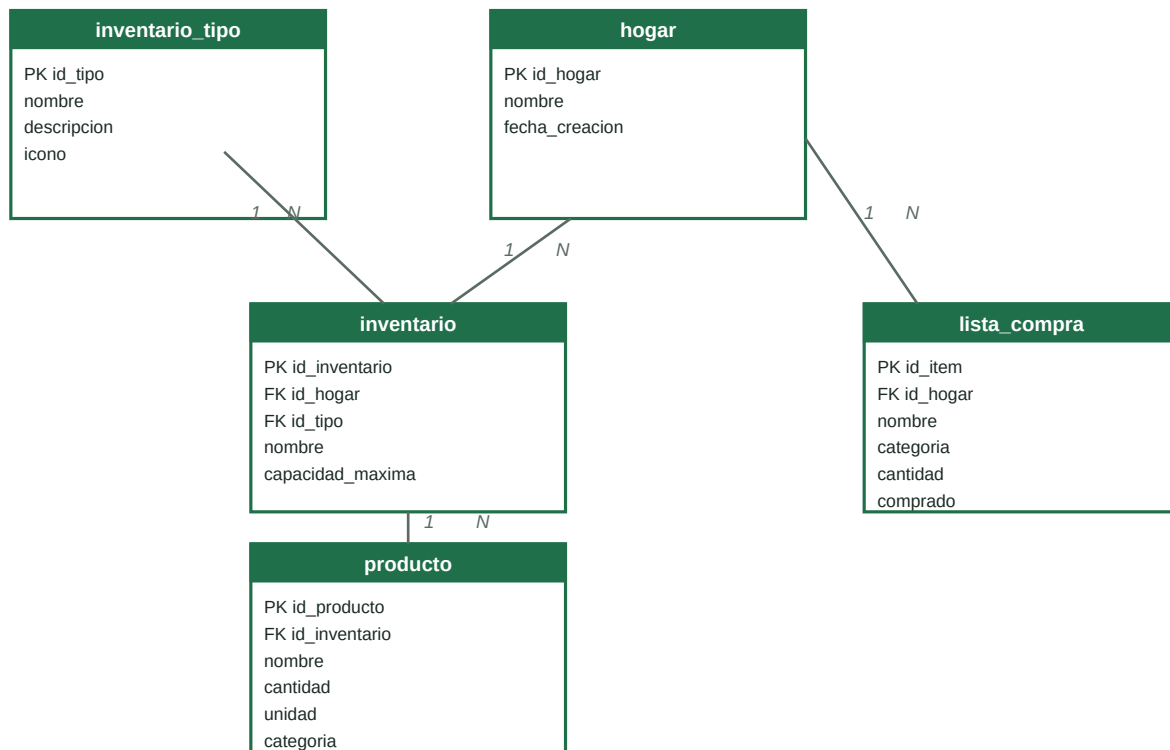
    private String nombre;
    private Integer capacidadMaxima;
    private LocalDateTime fechaCreacion;

    // Lado "1" de la relacion 1:N inventario -> producto.
    @OneToMany(mappedBy = "inventario", cascade = CascadeType.ALL, orphanRemoval = true)
    private List<ProductoBd> productos = new ArrayList<>();
}

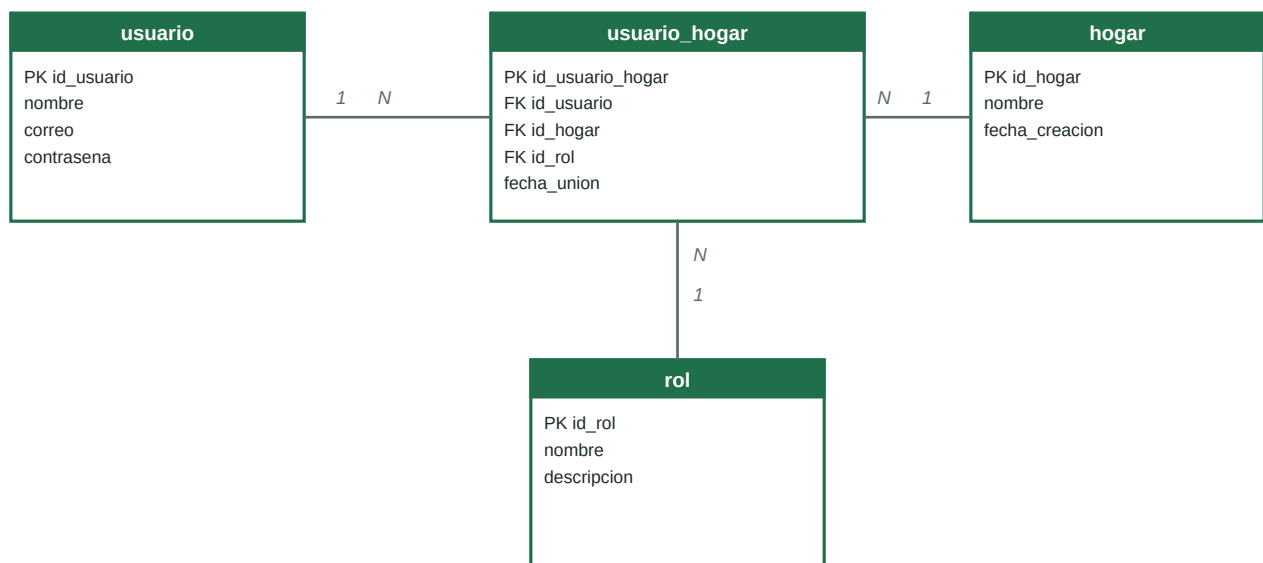
```

2.4 Diagramas entidad-relación

El siguiente diagrama resume el núcleo de relaciones 1:N de la aplicación (“1” en el origen de la flecha, “N” en el destino):



El siguiente diagrama muestra la relación N:M usuario<->hogar a través de la tabla intermedia usuario_hogar, con rol como catálogo del atributo propio de la asociación:



3. Arquitectura de la API REST

La API vive en el paquete `dgtic.core.rest`, separado de los controladores Thymeleaf existentes (`dgtic.core.controller`), y sigue una arquitectura en capas clásica:

Paquete	Responsabilidad
<code>dgtic.core.rest.controller</code>	Controladores <code>@RestController</code> : reciben la petición HTTP, validan con <code>@Valid</code> y devuelven <code>ResponseEntity<DTO></code> .
<code>dgtic.core.rest.service</code>	Reglas de negocio y acceso a datos vía Spring Data JPA (<code>JpaRepository</code>) sobre las entidades compartidas con la app web.
<code>dgtic.core.rest.mapper</code>	Clases <code>@Component</code> que convierten Entidad $\leftrightarrow$ DTO (Request/Response).
<code>dgtic.core.rest.dto</code>	DTO de entrada (<code>RequestDTO</code> , con <code>jakarta.validation</code>) y de salida (<code>ResponseDTO</code>) por recurso.
<code>dgtic.core.rest.exception</code>	Excepciones de negocio (404/400/409) y el <code>@RestControllerAdvice</code> que las traduce a respuestas HTTP uniformes.

Todas las rutas de la API cuelgan de `/api/v1`. El filtro de sesión (`SesionInterceptor`), que redirige a `/login` a cualquier petición sin sesión activa, se configuró para excluir `/api/**`, de modo que la API pueda probarse directamente con Postman o curl sin necesidad de iniciar sesión en la aplicación web.

```
@Configuration
public class WebConfig implements WebMvcConfigurer {

    @Override
    public void addInterceptors(InterceptorRegistry registry) {
        registry.addInterceptor(new SesionInterceptor())
            .addPathPatterns("/**")
            .excludePathPatterns(
                "/", "/login", "/registro",
                "/css/**", "/js/**", "/img/**",
                "/webjars/**", "/favicon.ico",
                "/api/**" // la API REST se prueba sin sesion (Postman/curl)
            );
    }
}
```

3.1 Controladores e inyección de dependencias

La API expone 7 controladores, cada uno responsable de un elemento funcional del modelo:

Controlador	Recurso / relación que expone
<code>HogarRestController</code>	CRUD de hogar; 1:N hogar->inventario y hogar->lista_compra; lado "hogar" del N:M usuario<->hogar.
<code>InventarioRestController</code>	CRUD de inventario; 1:N inventario->producto.
<code>ProductoRestController</code>	CRUD de producto (creación anidada o "plana").
<code>ListaCompraRestController</code>	CRUD de items de la lista de compras.
<code>InventarioTipoRestController</code>	CRUD del catálogo de tipos de inventario.

Controlador	Recurso / relación que expone
UsuarioRestController	CRUD de usuario; lado "usuario" del N:M usuario<->hogar (asociar, consultar, eliminar asociación).
RolRestController	CRUD del catálogo de roles (atributo de la relación N:M).

Todos los controladores y servicios REST usan **inyección de dependencias por constructor** (no por campo): cada clase declara sus dependencias como `private final` y Lombok genera el constructor con `@RequiredArgsConstructor`, en vez de anotar cada campo con `@Autowired`. Esto hace explícitas las dependencias, permite marcarlas como inmutables y facilita las pruebas unitarias (se puede instanciar la clase pasando mocks al constructor, sin un contenedor Spring).

```
@RestController
@RequestMapping("/api/v1/usuarios")
@RequiredArgsConstructor // Lombok genera el constructor con los "final" de abajo
public class UsuarioRestController {

    private final UsuarioRestService usuarioService;
    private final MembresiaRestService membresiaService;
    private final UsuarioMapper usuarioMapper;
    private final MembresiaMapper membresiaMapper;

    // ... los metodos usan estas dependencias normalmente
}
```

4. Manejo global de errores

Toda la API responde los errores con una estructura estándar `ErrorDetail`, construida y devuelta desde un único `@RestControllerAdvice` (`GlobalExceptionHandler`), restringido con `basePackages` a los controladores REST para no interferir con las páginas Thymeleaf.

Estructura estandarizada de error:

```
@JsonInclude(JsonInclude.Include.NON_NULL)
public record ErrorDetail(
    LocalDateTime timestamp,
    int status,
    String error,
    String mensaje,
    String ruta,
    List<String> detalles // detalle de errores de validacion, o null
) { }
```

Excepciones de negocio propias:

```
public class RecursoNoEncontradoException extends RuntimeException { ... } // -> 404
public class SolicitudInvalidaException extends RuntimeException { ... } // -> 400
public class ConflictoIntegridadException extends RuntimeException { ... } // -> 409
```

Manejador global (fragmento representativo):

```

@RestControllerAdvice(basePackages = "dgtic.core.rest.controller")
public class GlobalExceptionHandler {

    @ExceptionHandler(RecursoNoEncontradoException.class)
    public ResponseEntity<ErrorDetail> manejarNoEncontrado(RecursoNoEncontradoException ex,
                                                         HttpServletRequest request) {
        ErrorDetail error = new ErrorDetail(HttpStatus.NOT_FOUND.value(),
                                             HttpStatus.NOT_FOUND.getReasonPhrase(), ex.getMessage(),
                                             request.getRequestURI());
        return ResponseEntity.status(HttpStatus.NOT_FOUND).body(error);
    }

    @ExceptionHandler(ConflictoIntegridadException.class)
    public ResponseEntity<ErrorDetail> manejarConflicto(ConflictoIntegridadException ex,
                                                         HttpServletRequest request) {
        ErrorDetail error = new ErrorDetail(HttpStatus.CONFLICT.value(),
                                             HttpStatus.CONFLICT.getReasonPhrase(), ex.getMessage(),
                                             request.getRequestURI());
        return ResponseEntity.status(HttpStatus.CONFLICT).body(error);
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ErrorDetail> manejarValidacion(MethodArgumentNotValidException ex,
                                                         HttpServletRequest request) {
        List<String> detalles = ex.getBindingResult().getFieldErrors().stream()
            .map(fe -> fe.getField() + ": " + fe.getDefaultMessage())
            .toList();
        ErrorDetail error = new ErrorDetail(HttpStatus.BAD_REQUEST.value(),
                                             HttpStatus.BAD_REQUEST.getReasonPhrase(), "Los datos enviados no son validos",
                                             request.getRequestURI(), detalles);
        return ResponseEntity.badRequest().body(error);
    }

    @ExceptionHandler(DataIntegrityViolationException.class)
    public ResponseEntity<ErrorDetail> manejarIntegridadBd(DataIntegrityViolationException ex,
                                                         HttpServletRequest request) {
        ErrorDetail error = new ErrorDetail(HttpStatus.CONFLICT.value(),
                                             HttpStatus.CONFLICT.getReasonPhrase(),
                                             "La operacion entra en conflicto con datos relacionados existentes",
                                             request.getRequestURI());
        return ResponseEntity.status(HttpStatus.CONFLICT).body(error);
    }

    // + manejadores para ConstraintViolationException, MethodArgumentTypeMismatchException,
    //   HttpMessageNotReadableException, MissingServletRequestParamException,
    //   SolicitudInvalidaException/IllegalArgumentException y Exception (fallback 500).
}

```

Escenarios de error cubiertos y su código HTTP:

Situación	Excepción	HTTP
Recurso inexistente (id que no existe)	RecursoNoEncontradoException	404 Not Found
Cuerpo JSON inválido o campo que no cumple @NotBlank/@NotNull/@Min, etc.	MethodArgumentNotValidException	400 Bad Request
Un {id} en la URL no es numérico	MethodArgumentTypeMismatchException	400 Bad Request
JSON mal formado en el cuerpo de la petición	HttpMessageNotReadableException	400 Bad Request
Regla de negocio inválida (p. ej. capacidad fuera de {20,40,80})	SolicitudInvalidaException	400 Bad Request

Situación	Excepción	HTTP
Conflicto de negocio (inventario lleno, capacidad menor a lo ocupado)	ConflictoIntegridadException	409 Conflict
Eliminar un catálogo (tipo de inventario) todavía referenciado	ConflictoIntegridadException	409 Conflict
Violación de restricción de la base de datos (llave foránea, único)	DataIntegrityViolationException	409 Conflict

4.1 Serialización JSON de relaciones bidireccionales

El mecanismo **principal** para controlar qué información viaja en cada respuesta es el par DTO + Mapper: cada endpoint devuelve un `ResponseDTO` plano, nunca la entidad JPA completa, así que las colecciones `@OneToMany` y las referencias `@ManyToOne` nunca llegan directamente al cliente. Aun así, las entidades llevan anotaciones de Jackson como defensa adicional ("por si algún día" una entidad se serializara sin pasar por su DTO, por ejemplo desde un endpoint nuevo o una traza de depuración):

Anotación	Dónde se usa	Por qué
<code>@JsonIgnore</code>	Casi todos los lados <code>@ManyToOne</code> (<code>InventarioBd.tipo</code> , <code>ProductoBd.inventario</code> , <code>ListaCompraBd.hogar</code> , los tres <code>@ManyToOne</code> de <code>UsuarioHogarBd</code>) y algunas colecciones (<code>HogarBd.listaCompra</code> , <code>InventarioBd.productos</code> , <code>InventarioTipoBd.inventarios</code>) y <code>UsuarioBd.contrasena</code> .	Excluye el campo por completo de cualquier serialización JSON. Es la opción más simple cuando no interesa exponer esa relación (o ese dato sensible) ni siquiera en un escenario de serialización directa.
<code>@JsonManagedReference</code> / <code>@JsonBackReference</code>	Par <code>HogarBd.inventarios</code> (Managed) / <code>InventarioBd.hogar</code> (Back).	Alternativa clásica de Jackson para romper ciclos <code>hogar->inventario->hogar->...</code> : el lado "padre" SI se serializa, el lado "hijo" que apunta de vuelta se omite. Se dejó un solo par como demostración explícita del mecanismo.
<code>@JsonIgnoreProperties({"hibernateLazyInitializer", "handler"})</code>	A nivel de clase en <code>HogarBd</code> , <code>InventarioTipoBd</code> , <code>UsuarioBd</code> y <code>RolBd</code> (entidades que son destino de una relación <code>@ManyToOne</code> LAZY).	Evita errores de serialización si alguna vez Jackson intenta serializar el proxy de Hibernate de una relación LAZY en vez del objeto real.

¿Y los Mixins? No se usaron: los Mixins sirven para añadir anotaciones de Jackson a una clase sin modificar su código fuente (típicamente porque es de una librería externa que no se puede editar). Como todas las entidades de este proyecto son propias, es más simple y explícito anotarlas directamente que mantener un registro de Mixins en paralelo.

5. Documentación de las operaciones de la API REST

A continuación se documenta cada operación expuesta por la API, agrupada por recurso/controlador. Para cada una se indica una descripción breve, el método HTTP y el endpoint, los códigos de respuesta posibles y el código Java del método del controlador que la implementa.

5.1 Recurso Hogar — `dgitic.core.rest.controller.HogarRestController`

Base: `/api/v1/hogares`. Expone el CRUD del hogar y, como sub-recursos, las relaciones 1:N hogar->inventario y hogar->lista_compra, además del lado “hogar” de la relación N:M usuario<->hogar.

Lista todos los hogares registrados.

GET `/api/v1/hogares`

Códigos de respuesta posibles: 200 OK

Código Java (controlador):

```
@GetMapping
public ResponseEntity<List<HogarResponseDTO>> listar() {
    List<HogarResponseDTO> hogares = hogarService.listarTodos().stream()
        .map(hogarMapper::toResponseDTO)
        .toList();
    return ResponseEntity.ok(hogares);
}
```

Consulta un hogar por su identificador.

GET `/api/v1/hogares/{id}`

Códigos de respuesta posibles: 200 OK · 404 Not Found

Código Java (controlador):

```
@GetMapping("/{id}")
public ResponseEntity<HogarResponseDTO> obtener(@PathVariable("id") Integer id) {
    HogarBd hogar = hogarService.obtenerPorId(id);
    return ResponseEntity.ok(hogarMapper.toResponseDTO(hogar));
}
```

Crea un nuevo hogar. Valida el DTO con `@Valid` (nombre obligatorio, 3-100 caracteres) y responde con el header Location apuntando al recurso creado.

POST `/api/v1/hogares`

Códigos de respuesta posibles: 201 Created · 400 Bad Request

Ejemplo de cuerpo (JSON) de la petición:

```
{
  "nombre": "Casa Rivera"
}
```

Código Java (controlador):

```
@PostMapping
public ResponseEntity<HogarResponseDTO> crear(@Valid @RequestBody HogarRequestDTO dto) {
    HogarBd creado = hogarService.crear(dto);
    URI ubicacion = ServletUriComponentsBuilder.fromCurrentRequest()
        .path("/{id}").buildAndExpand(creado.getIdHogar()).toUri();
    return ResponseEntity.created(ubicacion).body(hogarMapper.toResponseDTO(creado));
}
```

Actualiza el nombre de un hogar existente.

PUT /api/v1/hogares/{id}

Códigos de respuesta posibles: 200 OK · 404 Not Found · 400 Bad Request

Ejemplo de cuerpo (JSON) de la petición:

```
{
  "nombre": "Casa Rivera (editada)"
}
```

Código Java (controlador):

```
@PutMapping("/{id}")
public ResponseEntity<HogarResponseDTO> actualizar(@PathVariable("id") Integer id,
    @Valid @RequestBody HogarRequestDTO dto) {
    HogarBd actualizado = hogarService.actualizar(id, dto);
    return ResponseEntity.ok(hogarMapper.toResponseDTO(actualizado));
}
```

Elimina un hogar. Gracias a la relación 1:N (cascade=ALL, orphanRemoval=true) se eliminan también, en cascada, sus inventarios (y los productos de estos) y su lista de compras.

DELETE /api/v1/hogares/{id}

Códigos de respuesta posibles: 204 No Content · 404 Not Found

Código Java (controlador):

```
@DeleteMapping("/{id}")
public ResponseEntity<Void> eliminar(@PathVariable("id") Integer id) {
    hogarService.eliminar(id);
    return ResponseEntity.noContent().build();
}
```

Consulta los inventarios asociados a un hogar (lectura de la relación 1:N hogar->inventario).

GET /api/v1/hogares/{id}/inventarios

Códigos de respuesta posibles: 200 OK · 404 Not Found

Código Java (controlador):

```
@GetMapping("/{id}/inventarios")
public ResponseEntity<List<InventarioResponseDTO>> listarInventarios(@PathVariable("id") Integer
    id) {
    List<InventarioResponseDTO> inventarios = inventarioService.listarPorHogar(id).stream()
        .map(inventarioMapper::toResponseDTO)
        .toList();
    return ResponseEntity.ok(inventarios);
}
```


Crea un inventario asociado al hogar {id} (creación de información asociada mediante la relación 1:N). Valida que el hogar y el tipo de inventario existan.

POST /api/v1/hogares/{id}/inventarios

Códigos de respuesta posibles: 201 Created · 404 Not Found · 400 Bad Request

Ejemplo de cuerpo (JSON) de la petición:

```
{
  "nombre": "Refrigerador",
  "capacidadMaxima": 20,
  "idTipo": 1
}
```

Código Java (controlador):

```
@PostMapping("/{id}/inventarios")
public ResponseEntity<InventarioResponseDTO> crearInventario(@PathVariable("id") Integer id,
                                                             @Valid @RequestBody InventarioRequestDTO dto) {
    var creado = inventarioService.crearParaHogar(id, dto);
    URI ubicacion = ServletUriComponentsBuilder.fromCurrentContextPath()
        .path("/api/v1/inventarios/{idInventario}")
        .buildAndExpand(creado.getIdInventario()).toUri();
    return ResponseEntity.created(ubicacion).body(inventarioMapper.toResponseDTO(creado));
}
```

Consulta la lista de compras de un hogar (lectura de la relación 1:N hogar->lista_compra).

GET /api/v1/hogares/{id}/lista-compra

Códigos de respuesta posibles: 200 OK · 404 Not Found

Código Java (controlador):

```
@GetMapping("/{id}/lista-compra")
public ResponseEntity<List<ListaCompraResponseDTO>> listarListaCompra(@PathVariable("id")
    Integer id) {
    List<ListaCompraResponseDTO> items = listaCompraService.listarPorHogar(id).stream()
        .map(listaCompraMapper::toResponseDTO)
        .toList();
    return ResponseEntity.ok(items);
}
```

Agrega un producto a la lista de compras del hogar {id} (creación de información asociada mediante la relación 1:N).

POST /api/v1/hogares/{id}/lista-compra

Códigos de respuesta posibles: 201 Created · 404 Not Found · 400 Bad Request

Ejemplo de cuerpo (JSON) de la petición:

```
{
  "nombre": "Tortillas",
  "categoria": "Panadería",
  "cantidad": 1,
  "unidad": "kg"
}
```

Código Java (controlador):

```

@PostMapping("/{id}/lista-compra")
public ResponseEntity<ListaCompraResponseDTO> agregarAListaCompra(@PathVariable("id") Integer
    id,
                                @Valid @RequestBody ListaCompraRequestDTO dto) {
    var creado = listaCompraService.agregarParaHogar(id, dto);
    URI ubicacion = ServletUriComponentsBuilder.fromCurrentContextPath()
        .path("/api/v1/lista-compra/{idItem}")
        .buildAndExpand(creado.getIdItem()).toUri();
    return ResponseEntity.created(ubicacion).body(listaCompraMapper.toResponseDTO(creado));
}

```

Consulta los usuarios asociados a este hogar, con su rol en cada uno (lado “hogar” de la relación N:M usuario->hogar; ver también la sección 5.6).

GET /api/v1/hogares/{id}/usuarios

Códigos de respuesta posibles: 200 OK · 404 Not Found

Código Java (controlador):

```

@GetMapping("/{id}/usuarios")
public ResponseEntity<List<MembresiaResponseDTO>> listarUsuarios(@PathVariable("id") Integer id)
{
    List<MembresiaResponseDTO> miembros = membresiaService.listarUsuariosDeHogar(id).stream()
        .map(membresiaMapper::toResponseDTO)
        .toList();
    return ResponseEntity.ok(miembros);
}

```

5.2 Recurso Inventario — `dgctic.core.rest.controller.InventarioRestController`

Base: `/api/v1/inventarios`. Es a la vez el lado “N” de `hogar->inventario` y el lado “1” de `inventario->producto`.

Lista todos los inventarios de todos los hogares. El query param opcional `idHogar` filtra el resultado a los inventarios de un solo hogar.

GET `/api/v1/inventarios[?idHogar=]`

Códigos de respuesta posibles: 200 OK · 404 Not Found (si `idHogar` no existe)

Código Java (controlador):

```
@GetMapping
public ResponseEntity<List<InventarioResponseDTO>> listar(
    @RequestParam(value = "idHogar", required = false) Integer idHogar) {
    List<InventarioResponseDTO> inventarios = inventarioService.listar(idHogar).stream()
        .map(inventarioMapper::toResponseDTO)
        .toList();
    return ResponseEntity.ok(inventarios);
}
```

Consulta un inventario por su identificador.

GET `/api/v1/inventarios/{id}`

Códigos de respuesta posibles: 200 OK · 404 Not Found

Código Java (controlador):

```
@GetMapping("/{id}")
public ResponseEntity<InventarioResponseDTO> obtener(@PathVariable("id") Integer id) {
    InventarioBd inventario = inventarioService.obtenerPorId(id);
    return ResponseEntity.ok(inventarioMapper.toResponseDTO(inventario));
}
```

Actualiza nombre, capacidad máxima y/o tipo de un inventario. Rechaza con 409 si la nueva capacidad es menor a la cantidad de productos ya guardados, y con 400 si no pertenece al catálogo de tamaños permitidos (20, 40, 80).

PUT `/api/v1/inventarios/{id}`

Códigos de respuesta posibles: 200 OK · 404 Not Found · 400 Bad Request · 409 Conflict

Ejemplo de cuerpo (JSON) de la petición:

```
{
  "nombre": "Refrigerador",
  "capacidadMaxima": 40,
  "idTipo": 1
}
```

Código Java (controlador):

```
@PutMapping("/{id}")
public ResponseEntity<InventarioResponseDTO> actualizar(@PathVariable("id") Integer id,
    @Valid @RequestBody InventarioRequestDTO dto) {
    InventarioBd actualizado = inventarioService.actualizar(id, dto);
    return ResponseEntity.ok(inventarioMapper.toResponseDTO(actualizado));
}
```

Elimina un inventario. Gracias a la relación 1:N (cascade=ALL, orphanRemoval=true) elimina también, en cascada, los productos que contenga.

DELETE /api/v1/inventarios/{id}

Códigos de respuesta posibles: 204 No Content · 404 Not Found

Código Java (controlador):

```
@DeleteMapping("/{id}")
public ResponseEntity<Void> eliminar(@PathVariable("id") Integer id) {
    inventarioService.eliminar(id);
    return ResponseEntity.noContent().build();
}
```

Consulta los productos activos de un inventario (lectura de la relación 1:N inventario->producto).

GET /api/v1/inventarios/{id}/productos

Códigos de respuesta posibles: 200 OK · 404 Not Found

Código Java (controlador):

```
@GetMapping("/{id}/productos")
public ResponseEntity<List<ProductoResponseDTO>> listarProductos(@PathVariable("id") Integer id)
{
    List<ProductoResponseDTO> productos = inventarioService.listarProductos(id).stream()
        .map(productoMapper::toResponseDTO)
        .toList();
    return ResponseEntity.ok(productos);
}
```

Agrega un producto al inventario {id} (creación de información asociada mediante la relación 1:N). Rechaza la operación con 409 Conflict si el inventario ya alcanzó su capacidad máxima.

POST /api/v1/inventarios/{id}/productos

Códigos de respuesta posibles: 201 Created · 404 Not Found · 400 Bad Request · 409 Conflict

Nota: La validación de conflicto vive en `InventarioRestService.agregarProducto`: compara `productoRepository.countByInventario_IdInventarioAndActivoTrue(idInventario)` contra `inventario.getCapacidadMaxima()` y lanza `ConflictoIntegridadException` si ya está lleno.

Ejemplo de cuerpo (JSON) de la petición:

```
{
  "nombre": "Mermelada",
  "cantidad": 1,
  "unidad": "pza",
  "categoria": "Abarrotes"
}
```

Código Java (controlador):

```
@PostMapping("/{id}/productos")
public ResponseEntity<ProductoResponseDTO> agregarProducto(@PathVariable("id") Integer id,
    @Valid @RequestBody ProductoRequestDTO dto) {
    var creado = inventarioService.agregarProducto(id, dto);
    URI ubicacion = ServletUriComponentsBuilder.fromCurrentContextPath()
        .path("/api/v1/productos/{idProducto}")
        .buildAndExpand(creado.getIdProducto()).toUri();
    return ResponseEntity.created(ubicacion).body(productoMapper.toResponseDTO(creado));
}
```


5.3 Recurso Producto — `dgctic.core.rest.controller.ProductoRestController`

Base: `/api/v1/productos`. Lado “N” de la relación inventario->producto. Puede crearse de dos formas equivalentes: anidado bajo su inventario (POST `/inventarios/{id}/productos`, sección 5.2) o de forma “plana” indicando el inventario destino en el cuerpo (POST `/productos`, aquí abajo). Ambas reutilizan la misma regla de negocio de capacidad máxima.

Lista todos los productos activos de todos los hogares. Los query params opcionales `idHogar` e `idInventario` filtran el resultado (por hogar, por inventario, o por ambos a la vez, validando que el inventario pertenezca al hogar indicado).

GET `/api/v1/productos[?idHogar=][&idInventario;=]`

Códigos de respuesta posibles: 200 OK · 404 Not Found (si `idHogar/idInventario` no existen o no coinciden)

Código Java (controlador):

```
@GetMapping
public ResponseEntity<List<ProductoResponseDTO>> listar(
    @RequestParam(value = "idHogar", required = false) Integer idHogar,
    @RequestParam(value = "idInventario", required = false) Integer idInventario) {
    List<ProductoResponseDTO> productos = productoService.listar(idHogar, idInventario).stream()
        .map(productoMapper::toResponseDTO)
        .toList();
    return ResponseEntity.ok(productos);
}
```

Crea un producto indicando el inventario destino en el cuerpo (`idInventario`). Rechaza la operación con 409 Conflict si ese inventario ya alcanzó su capacidad máxima, igual que la variante anidada de la sección 5.2.

POST `/api/v1/productos`

Códigos de respuesta posibles: 201 Created · 404 Not Found (`idInventario` no existe) · 400 Bad Request · 409 Conflict

Ejemplo de cuerpo (JSON) de la petición:

```
{
  "idInventario": 1,
  "nombre": "Mermelada",
  "cantidad": 1,
  "unidad": "pza",
  "categoria": "Abarrotes"
}
```

Código Java (controlador):

```
@PostMapping
public ResponseEntity<ProductoResponseDTO> crear(@Valid @RequestBody ProductoCreatedDTO dto) {
    ProductoBd creado = inventarioService.agregarProducto(dto.getIdInventario(),
        productoMapper.toRequestDTO(dto));
    URI ubicacion = ServletUriComponentsBuilder.fromCurrentRequest()
        .path("/{id}").buildAndExpand(creado.getIdProducto()).toUri();
    return ResponseEntity.created(ubicacion).body(productoMapper.toResponseDTO(creado));
}
```

Consulta un producto por su identificador.

GET `/api/v1/productos/{id}`

Códigos de respuesta posibles: 200 OK · 404 Not Found

Código Java (controlador):

```
@GetMapping("/{id}")
public ResponseEntity<ProductoResponseDTO> obtener(@PathVariable("id") Integer id) {
    ProductoBd producto = productoService.obtenerPorId(id);
    return ResponseEntity.ok(productoMapper.toResponseDTO(producto));
}
```

Actualiza nombre, cantidad, unidad y categoría de un producto.

PUT `/api/v1/productos/{id}`

Códigos de respuesta posibles: 200 OK · 404 Not Found · 400 Bad Request

Ejemplo de cuerpo (JSON) de la petición:

```
{
  "nombre": "Mermelada de fresa",
  "cantidad": 2,
  "unidad": "pza",
  "categoria": "Abarrotes"
}
```

Código Java (controlador):

```
@PutMapping("/{id}")
public ResponseEntity<ProductoResponseDTO> actualizar(@PathVariable("id") Integer id,
    @Valid @RequestBody ProductoRequestDTO dto) {
    ProductoBd actualizado = productoService.actualizar(id, dto);
    return ResponseEntity.ok(productoMapper.toResponseDTO(actualizado));
}
```

Elimina un producto.

DELETE `/api/v1/productos/{id}`

Códigos de respuesta posibles: 204 No Content · 404 Not Found

Código Java (controlador):

```
@DeleteMapping("/{id}")
public ResponseEntity<Void> eliminar(@PathVariable("id") Integer id) {
    productoService.eliminar(id);
    return ResponseEntity.noContent().build();
}
```

5.4 Recurso Lista de compras — `dgctic.core.rest.controller.ListaCompraRestController`

Base: `/api/v1/lista-compra`. Lado “N” de la relación hogar->lista_compra; su creación vive en `HogarRestController`.

Lista todos los items de la lista de compras de todos los hogares (útil para ver de un vistazo todo lo pendiente por comprar). El query param opcional `idHogar` filtra el resultado a los items de un solo hogar.

GET `/api/v1/lista-compra[?idHogar=]`

Códigos de respuesta posibles: 200 OK · 404 Not Found (si `idHogar` no existe)

Código Java (controlador):

```

@GetMapping
public ResponseEntity<List<ListaCompraResponseDTO>> listar(
    @RequestParam(value = "idHogar", required = false) Integer idHogar) {
    List<ListaCompraResponseDTO> items = listaCompraService.listar(idHogar).stream()
        .map(listaCompraMapper::toResponseDTO)
        .toList();
    return ResponseEntity.ok(items);
}

```

Consulta un ítem de la lista de compras por su identificador.

GET `/api/v1/lista-compra/{id}`

Códigos de respuesta posibles: 200 OK · 404 Not Found

Código Java (controlador):

```

@GetMapping("/{id}")
public ResponseEntity<ListaCompraResponseDTO> obtener(@PathVariable("id") Integer id) {
    ListaCompraBd item = listaCompraService.obtenerPorId(id);
    return ResponseEntity.ok(listaCompraMapper.toResponseDTO(item));
}

```

Actualiza un ítem de la lista de compras (nombre, categoría, cantidad, unidad y, opcionalmente, si ya fue comprado).

PUT `/api/v1/lista-compra/{id}`

Códigos de respuesta posibles: 200 OK · 404 Not Found · 400 Bad Request

Ejemplo de cuerpo (JSON) de la petición:

```

{
  "nombre": "Tortillas",
  "categoria": "Panadería",
  "cantidad": 2,
  "unidad": "kg",
  "comprado": true
}

```

Código Java (controlador):

```

@PutMapping("/{id}")
public ResponseEntity<ListaCompraResponseDTO> actualizar(@PathVariable("id") Integer id,
    @Valid @RequestBody ListaCompraRequestDTO dto) {
    ListaCompraBd actualizado = listaCompraService.actualizar(id, dto);
    return ResponseEntity.ok(listaCompraMapper.toResponseDTO(actualizado));
}

```

Elimina un ítem de la lista de compras.

DELETE `/api/v1/lista-compra/{id}`

Códigos de respuesta posibles: 204 No Content · 404 Not Found

Código Java (controlador):

```

@DeleteMapping("/{id}")
public ResponseEntity<Void> eliminar(@PathVariable("id") Integer id) {
    listaCompraService.eliminar(id);
    return ResponseEntity.noContent().build();
}

```


5.5 Catálogo de tipos de inventario — dgtic.core.rest.controller.InventarioTipoRestController

Base: /api/v1/tipos-inventario. Lado “1” de la relación 1:N con Inventario; permite demostrar el conflicto de integridad 409 al intentar borrar un tipo todavía referenciado.

Lista el catálogo completo de tipos de inventario.

GET /api/v1/tipos-inventario

Códigos de respuesta posibles: 200 OK

Código Java (controlador):

```
@GetMapping
public ResponseEntity<List<InventarioTipoResponseDTO>> listar() {
    List<InventarioTipoResponseDTO> tipos = tipoService.listarTodos().stream()
        .map(tipoMapper::toResponseDTO)
        .toList();
    return ResponseEntity.ok(tipos);
}
```

Consulta un tipo de inventario por su identificador.

GET /api/v1/tipos-inventario/{id}

Códigos de respuesta posibles: 200 OK · 404 Not Found

Código Java (controlador):

```
@GetMapping("/{id}")
public ResponseEntity<InventarioTipoResponseDTO> obtener(@PathVariable("id") Integer id) {
    InventarioTipoBd tipo = tipoService.obtenerPorId(id);
    return ResponseEntity.ok(tipoMapper.toResponseDTO(tipo));
}
```

Crea un nuevo tipo de inventario en el catálogo.

POST /api/v1/tipos-inventario

Códigos de respuesta posibles: 201 Created · 400 Bad Request

Ejemplo de cuerpo (JSON) de la petición:

```
{
  "nombre": "Congelador",
  "descripcion": "Almacenamiento a muy baja temperatura para congelados",
  "icono": "bi-snow"
}
```

Código Java (controlador):

```
@PostMapping
public ResponseEntity<InventarioTipoResponseDTO> crear(@Valid @RequestBody
    InventarioTipoRequestDTO dto) {
    InventarioTipoBd creado = tipoService.crear(dto);
    URI ubicacion = ServletUriComponentsBuilder.fromCurrentRequest()
        .path("/{id}").buildAndExpand(creado.getIdTipo()).toUri();
    return ResponseEntity.created(ubicacion).body(tipoMapper.toResponseDTO(creado));
}
```

Actualiza nombre, descripción e ícono de un tipo de inventario.

PUT /api/v1/tipos-inventario/{id}

Códigos de respuesta posibles: 200 OK · 404 Not Found · 400 Bad Request

Ejemplo de cuerpo (JSON) de la petición:

```
{
  "nombre": "Refrigerador",
  "descripcion": "Almacenamiento refrigerado para perecederos",
  "icono": "bi-snow2"
}
```

Código Java (controlador):

```
@PutMapping("/{id}")
public ResponseEntity<InventarioTipoResponseDTO> actualizar(@PathVariable("id") Integer id,
                                                           @Valid @RequestBody InventarioTipoRequestDTO
                                                           dto) {
    InventarioTipoBd actualizado = tipoService.actualizar(id, dto);
    return ResponseEntity.ok(tipoMapper.toResponseDTO(actualizado));
}
```

Elimina un tipo de inventario del catálogo. Rechaza la operación con 409 Conflict si todavía existen inventarios (relación 1:N) que lo referencian.

DELETE /api/v1/tipos-inventario/{id}

Códigos de respuesta posibles: 204 No Content · 404 Not Found · 409 Conflict

Nota: La regla vive en `InventarioTipoRestService.eliminar`: consulta `inventarioRepository.countByTipo_IdTipo(idTipo)` y, si es mayor a cero, lanza `ConflictIntegridadException` antes de borrar.

Código Java (controlador):

```
@DeleteMapping("/{id}")
public ResponseEntity<Void> eliminar(@PathVariable("id") Integer id) {
    tipoService.eliminar(id);
    return ResponseEntity.noContent().build();
}
```

5.6 Recurso Usuario — `dgitic.core.rest.controller.UsuarioRestController`

Base: `/api/v1/usuarios`. Además del CRUD propio, expone el lado “usuario” de la relación N:M usuario->hogar (tabla intermedia usuario_hogar): asociar, consultar y eliminar la asociación con un hogar.

Lista todos los usuarios registrados (nunca incluye la contraseña).

GET `/api/v1/usuarios`

Códigos de respuesta posibles: 200 OK

Código Java (controlador):

```
@GetMapping
public ResponseEntity<List<UsuarioResponseDTO>> listar() {
    List<UsuarioResponseDTO> usuarios = usuarioService.listarTodos().stream()
        .map(usuarioMapper::toResponseDTO)
        .toList();
    return ResponseEntity.ok(usuarios);
}
```

Consulta un usuario por su identificador.

GET `/api/v1/usuarios/{id}`

Códigos de respuesta posibles: 200 OK · 404 Not Found

Código Java (controlador):

```
@GetMapping("/{id}")
public ResponseEntity<UsuarioResponseDTO> obtener(@PathVariable("id") Integer id) {
    UsuarioBd usuario = usuarioService.obtenerPorId(id);
    return ResponseEntity.ok(usuarioMapper.toResponseDTO(usuario));
}
```

Crea un usuario. Rechaza con 409 Conflict si el correo ya está registrado.

POST `/api/v1/usuarios`

Códigos de respuesta posibles: 201 Created · 400 Bad Request · 409 Conflict

Ejemplo de cuerpo (JSON) de la petición:

```
{
  "nombre": "Ana Torres",
  "correo": "ana.torres@correo.com",
  "contrasena": "clave123"
}
```

Código Java (controlador):

```
@PostMapping
public ResponseEntity<UsuarioResponseDTO> crear(@Valid @RequestBody UsuarioRequestDTO dto) {
    UsuarioBd creado = usuarioService.crear(dto);
    URI ubicacion = ServletUriComponentsBuilder.fromCurrentRequest()
        .path("/{id}").buildAndExpand(creado.getIdUsuario()).toUri();
    return ResponseEntity.created(ubicacion).body(usuarioMapper.toResponseDTO(creado));
}
```

Actualiza nombre, correo y/o si el usuario sigue activo. No permite cambiar la contraseña (ese es un flujo aparte en la aplicación web).

PUT `/api/v1/usuarios/{id}`

Códigos de respuesta posibles: 200 OK · 404 Not Found · 400 Bad Request · 409 Conflict

Ejemplo de cuerpo (JSON) de la petición:

```
{
  "nombre": "Ana Torres López",
  "correo": "ana.torres@correo.com",
  "activo": true
}
```

Código Java (controlador):

```
@PutMapping("/{id}")
public ResponseEntity<UsuarioResponseDTO> actualizar(@PathVariable("id") Integer id,
                                                    @Valid @RequestBody UsuarioUpdateDTO dto) {
    UsuarioBd actualizado = usuarioService.actualizar(id, dto);
    return ResponseEntity.ok(usuarioMapper.toResponseDTO(actualizado));
}
```

Elimina un usuario. No hace cascada sobre usuario_hogar a propósito: si el usuario todavía tiene membresías, la llave foránea real de la relación N:M rechaza el borrado.

DELETE /api/v1/usuarios/{id}

Códigos de respuesta posibles: 204 No Content · 404 Not Found · 409 Conflict

Nota: El 409 lo produce `DataIntegrityViolationException` (violación de la FK `id_usuario` en `usuario_hogar`), capturada por el manejador global. Hay que eliminar antes la asociación con `DELETE /usuarios/{id}/hogares/{idHogar}`.

Código Java (controlador):

```
@DeleteMapping("/{id}")
public ResponseEntity<Void> eliminar(@PathVariable("id") Integer id) {
    usuarioService.eliminar(id);
    return ResponseEntity.noContent().build();
}
```

Relación N:M usuario <-> hogar (lado “usuario”)

Consulta los hogares asociados a este usuario, con su rol en cada uno (lado “usuario” de la relación N:M).

GET /api/v1/usuarios/{idUsuario}/hogares

Códigos de respuesta posibles: 200 OK · 404 Not Found

Código Java (controlador):

```
@GetMapping("/{idUsuario}/hogares")
public ResponseEntity<List<MembresiaResponseDTO>> listarHogares(
    @PathVariable("idUsuario") @Positive Integer idUsuario) {
    List<MembresiaResponseDTO> hogares =
        membresiaService.listarHogaresDeUsuario(idUsuario).stream()
            .map(membresiaMapper::toResponseDTO)
            .toList();
    return ResponseEntity.ok(hogares);
}
```

Asocia (crea la relación N:M) al usuario con el hogar, con el rol indicado en el cuerpo. Rechaza con 409 si ya existe esa asociación activa, o si el usuario ya pertenece a otro hogar activo (regla de negocio de NUBOM).

POST /api/v1/usuarios/{idUsuario}/hogares/{idHogar}

Códigos de respuesta posibles: 201 Created · 404 Not Found · 400 Bad Request · 409 Conflict

Ejemplo de cuerpo (JSON) de la petición:

```
{
  "idRol": 2
}
```

Código Java (controlador):

```
@PostMapping("/{idUsuario}/hogares/{idHogar}")
public ResponseEntity<MembresiaResponseDTO> asociarConHogar(
    @PathVariable("idUsuario") @Positive Integer idUsuario,
    @PathVariable("idHogar") @Positive Integer idHogar,
    @Valid @RequestBody MembresiaRequestDTO dto) {
    var creada = membresiaService.asociar(idUsuario, idHogar, dto);
    URI ubicacion = ServletUriComponentsBuilder.fromCurrentContextPath()
        .path("/api/v1/usuarios/{idUsuario}/hogares")
        .buildAndExpand(idUsuario).toUri();
    return ResponseEntity.created(ubicacion).body(membresiaMapper.toResponseDTO(creada));
}
```

Elimina la asociación (relación N:M) entre el usuario y el hogar.

DELETE /api/v1/usuarios/{idUsuario}/hogares/{idHogar}

Códigos de respuesta posibles: 204 No Content · 404 Not Found

Código Java (controlador):

```
@DeleteMapping("/{idUsuario}/hogares/{idHogar}")
public ResponseEntity<Void> eliminarAsociacion(
    @PathVariable("idUsuario") @Positive Integer idUsuario,
    @PathVariable("idHogar") @Positive Integer idHogar) {
    membresiaService.eliminarAsociacion(idUsuario, idHogar);
    return ResponseEntity.noContent().build();
}
```

5.7 Catálogo de roles — `dgctic.core.rest.controller.RolRestController`

Base: `/api/v1/roles`. Catálogo del atributo propio de la relación N:M usuario<->hogar (PROPIETARIO, FAMILIAR). Permite demostrar el 409 Conflict por integridad: no se puede borrar un rol con membresías activas.

Lista el catálogo completo de roles.

GET `/api/v1/roles`

Códigos de respuesta posibles: 200 OK

Código Java (controlador):

```
@GetMapping
public ResponseEntity<List<RolResponseDTO>> listar() {
    List<RolResponseDTO> roles = rolService.listarTodos().stream()
        .map(r -> rolMapper.toResponseDTO(r,
            rolService.contarMembresiasActivas(r.getIdRol())))
        .toList();
    return ResponseEntity.ok(roles);
}
```

Consulta un rol por su identificador.

GET `/api/v1/roles/{id}`

Códigos de respuesta posibles: 200 OK · 404 Not Found

Código Java (controlador):

```
@GetMapping("/{id}")
public ResponseEntity<RolResponseDTO> obtener(@PathVariable("id") Integer id) {
    RolBd rol = rolService.obtenerPorId(id);
    return ResponseEntity.ok(rolMapper.toResponseDTO(rol,
        rolService.contarMembresiasActivas(id)));
}
```

Crea un nuevo rol en el catálogo.

POST `/api/v1/roles`

Códigos de respuesta posibles: 201 Created · 400 Bad Request

Ejemplo de cuerpo (JSON) de la petición:

```
{
  "nombre": "INVITADO",
  "descripcion": "Acceso de solo lectura al hogar"
}
```

Código Java (controlador):

```
@PostMapping
public ResponseEntity<RolResponseDTO> crear(@Valid @RequestBody RolRequestDTO dto) {
    RolBd creado = rolService.crear(dto);
    URI ubicacion = ServletUriComponentsBuilder.fromCurrentRequest()
        .path("/{id}").buildAndExpand(creado.getIdRol()).toUri();
    return ResponseEntity.created(ubicacion).body(rolMapper.toResponseDTO(creado, 0));
}
```

Actualiza nombre y descripción de un rol.

PUT `/api/v1/roles/{id}`

Códigos de respuesta posibles: 200 OK · 404 Not Found · 400 Bad Request

Ejemplo de cuerpo (JSON) de la petición:

```
{
  "nombre": "FAMILIAR",
  "descripcion": "Puede consultar y actualizar el inventario del hogar"
}
```

Código Java (controlador):

```
@PutMapping("/{id}")
public ResponseEntity<RolResponseDTO> actualizar(@PathVariable("id") Integer id,
                                                @Valid @RequestBody RolRequestDTO dto) {
    RolBd actualizado = rolService.actualizar(id, dto);
    return ResponseEntity.ok(rolMapper.toResponseDTO(actualizado,
        rolService.contarMembresiasActivas(id)));
}
```

Elimina un rol del catálogo. Rechaza la operación con 409 Conflict si todavía existen membresías activas (usuario_hogar) que lo referencian.

DELETE /api/v1/roles/{id}

Códigos de respuesta posibles: 204 No Content · 404 Not Found · 409 Conflict

Nota: La regla vive en RolRestService.eliminar: consulta usuarioHogarRepository.countByRol_IdRolAndActivoTrue(idRol) y, si es mayor a cero, lanza ConflictIntegridadException antes de borrar.

Código Java (controlador):

```
@DeleteMapping("/{id}")
public ResponseEntity<Void> eliminar(@PathVariable("id") Integer id) {
    rolService.eliminar(id);
    return ResponseEntity.noContent().build();
}
```

6. Evidencia de pruebas (curl) y escenarios exigidos

A continuación se muestran, con curl, los siete escenarios de prueba exigidos para el entregable, ejecutados contra la API en ejecución (`http://localhost:8080/api/v1`). Las mismas pruebas pueden repetirse en Postman importando cada petición con el método, la URL y el cuerpo JSON indicados.

200 OK — Consulta exitosa

GET `/api/v1/hogares`

```
$ curl -s http://localhost:8080/api/v1/hogares

HTTP 200
[{"idHogar":1,"nombre":"Casa Luna","fechaCreacion":"2026-09-04T09:47:06",
  "totalInventarios":2,"totalListaCompra":4}]
```

201 Created — Creación de un recurso

POST `/api/v1/hogares`

```
$ curl -s -i -X POST http://localhost:8080/api/v1/hogares \
  -H "Content-Type: application/json" -d '{"nombre":"Casa Demo"}'

HTTP/1.1 201
Location: http://localhost:8080/api/v1/hogares/3
{"idHogar":3,"nombre":"Casa Demo","fechaCreacion":"2026-09-04T09:47:54.31",
  "totalInventarios":0,"totalListaCompra":0}
```

200 OK — Actualización de un recurso

PUT `/api/v1/hogares/3`

```
$ curl -s -X PUT http://localhost:8080/api/v1/hogares/3 \
  -H "Content-Type: application/json" -d '{"nombre":"Casa Demo Editada"}'

HTTP 200
{"idHogar":3,"nombre":"Casa Demo Editada", ...}
```

204 No Content — Eliminación exitosa (con cascada 1:N)

DELETE `/api/v1/hogares/3`

```
$ curl -s -o /dev/null -w "HTTP %{http_code}\n" \
  -X DELETE http://localhost:8080/api/v1/hogares/3

HTTP 204
(el inventario "Alacena Demo" hijo de este hogar tambien desaparece:
GET /api/v1/inventarios/3 responde 404 despues de este DELETE)
```


404 Not Found — Recurso inexistente

GET /api/v1/hogares/9999

```
$ curl -s http://localhost:8080/api/v1/hogares/9999

HTTP 404
{"timestamp":"2026-09-04T09:47:54.63","status":404,"error":"Not Found",
 "mensaje":"No existe un hogar con id 9999","ruta":"/api/v1/hogares/9999"}
```

400 Bad Request — Solicitud/parámetro inválido

POST /api/v1/hogares (nombre vacío) y GET /api/v1/hogares/abc

```
$ curl -s -X POST http://localhost:8080/api/v1/hogares \
-H "Content-Type: application/json" -d '{"nombre":""}'
HTTP 400
{"status":400,"error":"Bad Request","mensaje":"Los datos enviados no son validos",
 "detalles":["nombre: El nombre del hogar es obligatorio",
             "nombre: El nombre debe tener entre 3 y 100 caracteres"]}

$ curl -s http://localhost:8080/api/v1/hogares/abc
HTTP 400
{"status":400,"error":"Bad Request",
 "mensaje":"El parametro 'id' debe ser de tipo Integer"}
```

409 Conflict — Conflicto de integridad / regla de negocio

POST /api/v1/inventarios/{id}/productos (inventario lleno) y DELETE /api/v1/tipos-inventario/{id} (en uso)

```
$ curl -s -X POST http://localhost:8080/api/v1/inventarios/3/productos \
-H "Content-Type: application/json" \
-d '{"nombre":"ItemExtra","cantidad":1,"unidad":"pza","categoria":"Otros"}'
HTTP 409
{"status":409,"error":"Conflict",
 "mensaje":"\"Alacena Demo\" ya esta lleno (20 objetos max.)"}

$ curl -s -X DELETE http://localhost:8080/api/v1/tipos-inventario/2
HTTP 409
{"status":409,"error":"Conflict",
 "mensaje":"No se puede eliminar el tipo \"Alacena\": esta siendo usado por 2
            inventario(s)"}

```

Nota: las pruebas se ejecutaron con la aplicación corriendo localmente (`./mvnw spring-boot:run`) contra una base MariaDB local, con el esquema recreado automáticamente en cada arranque (`ddl-auto=create`) y sembrado con los datos de `data.sql`. La colección puede reproducirse en Postman creando una petición por cada fila de las tablas de la sección 5, usando `http://localhost:8080` como host base.